

Campus Lost & Found Matcher

A terminal application for campus reports and matching

Student Name	Yatharth Singh
Registration No.	25BAI10657
Course	Programming in Java
Report Date	17 September 2026
GitHub Repository	https://github.com/YatharthSingh-VIT/campus-lost-found-matcher

Structured reports, ranked suggestions, and local CSV storage.
Ten application source files, one test source file, and three functional modules.

1. Introduction

A student loses a bottle in the library. Another student finds it and leaves it at a help desk. Both reports may exist, yet the students still need a way to connect them. Campus Lost & Found Matcher records the details in one place and suggests reports that may describe the same item.

The application runs entirely in a Java terminal. A student or help-desk operator can record an item, search existing reports, review suggested matches, and update the report when the case is resolved. Records remain available after the application closes because they are saved in a local CSV file.

Three modules handle the work: Report Management, Matching Engine, and Search & Reporting. The implementation uses Java classes, inheritance, enums, collections, exception handling, dates, and file input/output. A JDK is sufficient to compile and run both the application and its tests.

The objectives are practical: capture consistent information, reduce the number of reports that must be compared manually, explain each match score, and keep open and resolved cases easy to distinguish.

Source code repository: <https://github.com/YatharthSingh-VIT/campus-lost-found-matcher>

2. Problem Statement

Lost-and-found messages are difficult to manage when they are spread across notice boards and group chats. Descriptions vary, older messages are hard to locate, and resolved cases may still appear active. This project addresses the record-keeping and comparison part of that problem.

The intended users are students and campus help-desk staff sharing one installation. Each report records whether an item was lost or found, along with its title, description, category, location, incident date, contact reference, and current status.

The scope includes adding, viewing, editing, deleting, resolving, and reopening reports. It also includes ranked suggestions, combined search filters, and basic statistics. Accounts, notifications, image matching, and communication between separate computers are outside this version.

The application assumes that incident dates are reasonably accurate and that the computer clock is correct. Its data folder must allow writes and atomic file replacement. A file lock allows only one application instance to work on a particular dataset at a time.

A successful result is a consistent set of records and useful suggestions for follow-up. A high score does not confirm ownership, and the application does not record a verified relationship between a lost report and a found report.

3. Functional Requirements

ID / Module	Required behaviour	Evidence
FR1 / Reports	Create LOST or FOUND with unique ID and saved data.	Service + CLI tests
FR2 / Reports	View details; edit while preserving ID and type.	Service + CLI tests
FR3 / Reports	Delete only after explicit confirmation.	CLI cancellation and deletion tests
FR4 / Reports	Resolve/reopen and persist the status.	Service + CLI tests
FR5 / Matching	Filter, score, threshold, and rank eligible pairs.	Matcher boundary tests
FR6 / Matching	Show point breakdown, day gap, and shared keywords.	Matcher tests + captured output
FR7 / Search	Combine text/category/location/type/status/date filters.	Service + CLI tests
FR8 / Reporting	Count reports by type, status, and category.	Service tests + demo output

Report forms accept single-line text, listed category and status values, and dates in YYYY-MM-DD format. Creating a report returns a unique ID. Searches return matching records; the matcher adds scores and their component values. Statistics display counts of report rows.

The main menu has five actions and an exit option. Longer lists pause after five records. Invalid entries can be corrected at the prompt. A completed action returns to the menu, while the end of input closes the application without saving an unfinished form.

4. Non-functional Requirements

Attribute	Acceptance target
Reliability	Reload preserves records. A failed save does not update service memory. A second app instance cannot obtain the same data lock.
Usability	Five home actions plus exit, five-record pages, clear prompts, and an ASCII/no-colour fallback. No formal user study has been conducted.
Maintainability	Ten production files separate UI, models, services, storage, and validation. Business logic can be tested without keyboard interaction.

Attribute	Acceptance target
Error handling	Reject invalid dates, text, and enum values; report malformed CSV rows with context and leave input unchanged.
Performance	Rank 10,000 bounded-length synthetic candidates for one source report in under 2 seconds on the development laptop.
Bounded scalability	Handle the synthetic 10,000-report matching case; whole-file rewrites and in-memory storage are documented limits.

These targets describe the intended behaviour of this local prototype. Automated tests check data handling and the main workflows. The timing result covers one matching run on the development laptop. Usability has not been measured in a student study, and the project has not been evaluated as a production service.

5. System Architecture

The system follows a single-installation, layered design with clear presentation, service, model, and storage boundaries, as shown in Figure 1.

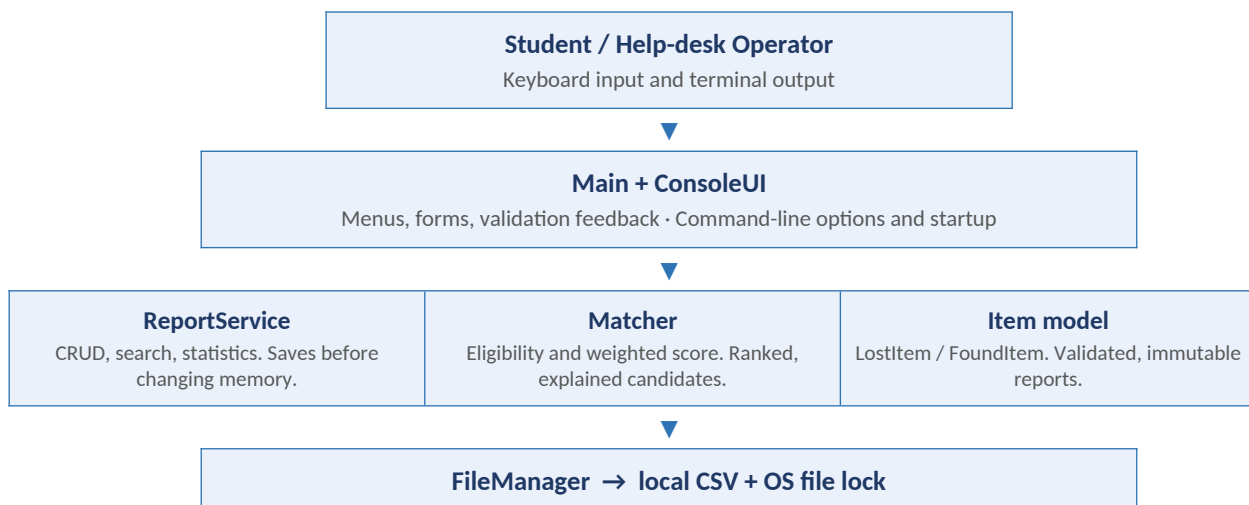


Figure 1. Architecture and local persistence boundary.

Main starts the application, selects the data file, and handles menu actions. ConsoleUI presents prompts and results. ReportService maintains the collection of reports and saves changes through FileManager. Matcher works with report objects to produce ranked suggestions.

Item defines the common report fields. LostItem and FoundItem provide the two report types. ValidationUtil checks inputs, and InvalidInputException represents errors that the user can correct. These responsibilities are separated so that storage and matching can be tested without navigating terminal menus.

6. Design Diagrams

6.1 Use Case Diagram

One shared-terminal actor; no login, remote messaging, or automatic ownership claim.

Actor	Supported use cases
Student / operator	Create lost / found report · View, edit, delete report · Resolve / reopen report · Find potential matches · Search reports · View statistics

Figure 2. The shared-terminal actor and supported use cases.

6.2 Workflow Diagram

Every confirmed mutation saves immediately; unresolved forms create no records.

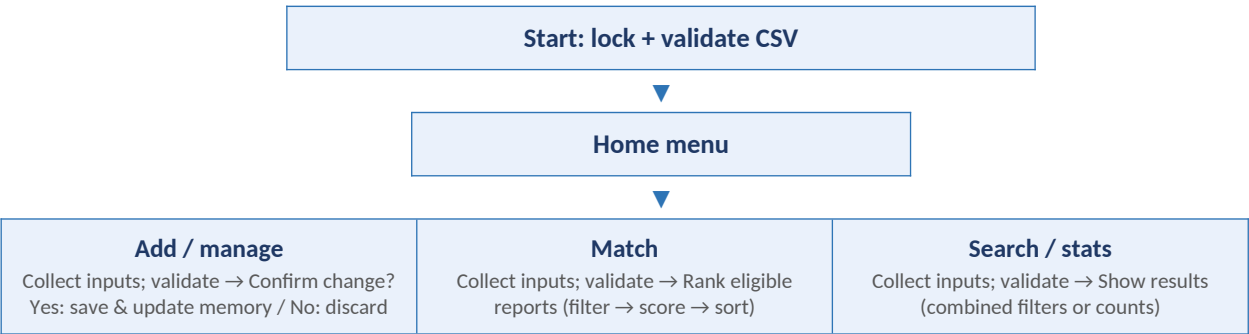


Figure 3. Startup, actions, validation, persistence, and exit.

- All completed actions return to Home. Invalid fields retry; failed operations show an error.
- Home: option 0 releases the lock and exits. End-of-input discards an unfinished form, releases the lock, and exits.
- A startup error explains the problem and exits with code 1 without overwriting damaged input.

6.3 Class Diagram

Representative members and dependencies; hollow arrows denote inheritance.

Class	Key members	Role
Main	main(args), run(ui, service), add(), manage()	Entry point, startup, menu dispatch
ConsoleUI	ask(), menu(), reports(), matches(), error(), heading()	Prompts, menus, formatted output

Class	Key members	Role
ValidationUtil	text(), date(), choice(), normalize()	Input validation and normalization
ReportService	items: List<Item>, add(), update(), delete(), setStatus(), search(), statistics()	In-memory collection, CRUD, search
Matcher	score(): Optional<Match>, findMatches(): List<Match>, THRESHOLD=60, MAX_DAYS=30	Eligibility filter and weighted scoring
FileManager	path, channel, lock, load(), save(), close() — implements AutoCloseable	CSV persistence, atomic replace, locking
Item (abstract)	id, type via getType(), title, category, date, status	Common report fields and behaviour
LostItem / FoundItem	getType() → LOST / FOUND	Concrete report subtypes
InvalidInputException	extends IllegalArgumentException	Recoverable validation errors

Item nests the Type, Category, and Status enums. Match is a record inside Matcher that stores a candidate and its score breakdown.

Figure 4. Ten production classes with representative dependencies and inheritance.

6.4 Sequence Diagram — Saving a New Report

Step	Interaction
1	ConsoleUI → Main: validated form + confirmation
2	Main → ReportService: add(report fields)
3	ReportService → FileManager: save(candidate list)
4	FileManager → filesystem: write + flush temporary file, then atomic replacement
5	Filesystem → FileManager: success
6	FileManager → ReportService: return; install list
7	ReportService → Main: saved item
8	Main → ConsoleUI: show ID and matches

If the write or the atomic move fails, an `IOException` is reported to Main, the error is shown, and the previous in-memory list is kept — the service publishes new state only after persistence succeeds.

Figure 5. The service publishes changed memory only after persistence succeeds.

6.5 ER Diagram and Storage Schema

One stored entity. Matching results are derived at request time and have no persistent relationship.

Entity	REPORT data/items.csv
id (PK)	string / UUID generated by the application
type	LOST FOUND
title	required, ≤ 80 characters
description	required, ≤ 500 characters
category	one of seven categories
location	required, ≤ 80 characters
date	ISO date, not in the future
contact	required, ≤ 120 characters
status	OPEN RESOLVED

Figure 6. A single stored Report entity with id as its primary key.

LOST and FOUND share the same CSV schema; the type field selects the Java subclass. No user table, match table, foreign keys, or invented relationships are stored. Match results are calculated when requested and are not stored as a separate entity.

Titles and locations allow up to 80 characters, descriptions 500, and contact references 120. Incident dates must be valid and cannot be in the future. CSV fields are quoted, with internal quotes doubled. Newlines and control characters are rejected so that each report occupies one row.

7. Design Decisions & Rationale

Decision	Reason and trade-off
Java CLI	Satisfies terminal execution and demonstrates course concepts. No GUI setup is

Decision	Reason and trade-off
	needed; rich visual interaction is outside scope.
Plain javac / java	No build-tool or test-framework dependencies. An argument file lists source paths explicitly; the JDK remains a prerequisite.
CSV + in-memory list	Readable storage and simple setup. Every mutation rewrites all reports, and searches scan memory — it is not a multi-user database.
Weighted heuristic score	Transparent, deterministic scoring. Weights and threshold are assumptions, not trained or empirically calibrated values.
Immutable reports	Edits construct validated replacements, which simplifies rollback and prevents silent changes to the service collection.
Two Item subtypes	Makes inheritance and polymorphic report types explicit. A single concrete class with a Type field would also be viable.
Atomic replacement + lock	Avoids overlapping app sessions and partial replacements on supported filesystems. Unsupported atomic moves produce an error.
Optional terminal styling	Restrained colour and borders aid readability; plain mode supports evaluators and redirected output.

Simple rules make the score easy to inspect, but they also have limits. Location matching ignores case and extra spaces; it does not know that two different place names may refer to the same building. The 30-day window and 60-point threshold are design choices whose usefulness would need to be measured against labelled campus reports.

8. Implementation Details

The application contains ten source files across six packages. Main reads the command-line options, opens FileManager, creates ReportService, and enters the menu loop. Without a data option, it uses data/items.csv; demo mode copies the fictional examples to a separate file on first use.

Report objects store their fields as private final values. Item.create constructs the appropriate lost or found subtype. ReportService assigns a UUID to each new report. An edit preserves the ID and type, while resolve and reopen actions change the status. Each operation prepares a replacement list, saves it, and only then updates the service.

Before scoring, Matcher excludes identical IDs, reports of the same type, resolved cases, and different categories. The found date must fall between the lost date and 30 days later, inclusive. Keywords are taken from the title and description, converted to lowercase, and stored as sets, so repeated words earn no extra points.


```
score = category + location + date + keywords category = 35 location = same normalized
location ? 25 : 0 date = round(25.0 * (30 - dayGap) / 30) keywords = unionSize ==
0 ? 0 : round(15.0 * sharedSize / unionSize)
```

A result is shown when its total reaches 60. Results are ordered by higher score, then shorter date gap, then candidate ID. Alongside the total, each Match record keeps the category, location, date, and keyword points, the day gap, and the shared keywords.

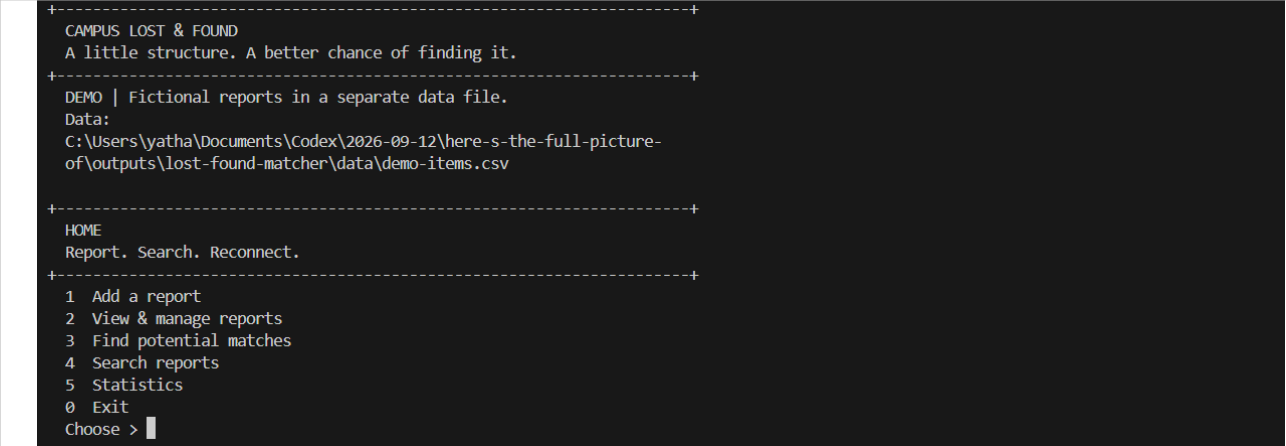
Search applies all supplied filters together and includes both ends of a date range. FileManager checks the CSV before returning any records. To save, it writes a neighbouring temporary file, flushes it, and requests an atomic replacement. The file lock and channel are closed with try-with-resources. Invalid CSV stops loading without clearing the original file.

9. Screenshots / Results

The figures below are real terminal screenshots captured on 18 September 2026 from a live run of the application (--demo --plain), walking through a full session: startup, adding a report, viewing saved reports, opening a report, searching, viewing statistics, and exiting.

9.1 Startup and home menu

The application starts in demo mode, loads the fictional dataset from data/demo-items.csv, and presents the home menu with five actions plus exit.



```
+-----+
| CAMPUS LOST & FOUND |
| A little structure. A better chance of finding it. |
+-----+
| DEMO | Fictional reports in a separate data file. |
| Data: |
| C:\Users\yatha\Documents\Codex\2026-09-12\here-s-the-full-picture- |
| of\outputs\lost-found-matcher\data\demo-items.csv |
+-----+
| HOME |
| Report. Search. Reconnect. |
+-----+
| 1 Add a report |
| 2 View & manage reports |
| 3 Find potential matches |
| 4 Search reports |
| 5 Statistics |
| 0 Exit |
| Choose > |
```

Figure 7. Startup banner, demo data path, and the home menu (options 1–5, 0 to exit).

9.2 Adding a report

Choosing option 1 opens the new-report form. A FOUND report for a green bottle at location "ab2" is entered and saved; the service assigns a UUID and reports that no existing candidate currently reaches the 60/100 threshold.

```

Choose > 1

+-----+
NEW REPORT
Use a help-desk reference if you prefer not to store personal
contact details.
+-----+
[LOST, FOUND]
Type > found
Title > bottle
Description > green in colour
[ELECTRONICS, ID_CARD, KEYS, BOOKS, CLOTHING, BAGS, OTHER]
Category > other
Location > ab2
Incident date (YYYY-MM-DD) > 2026-09-18
Contact / help-desk reference > 7258763868
Save this report? (type yes) > yes
Saved. Report ID: 9524a7d4-b17a-4f81-bb02-fb62b118ec8c
No candidates reached 60/100. Try again when new reports arrive.

```

Figure 8. Adding a FOUND report through option 1; the report is saved with a generated ID.

9.3 Viewing and managing reports

Option 2 lists all saved reports, five at a time. After the newly added report, the dataset holds eight reports spanning LOST/FOUND types and OPEN/RESOLVED statuses across categories such as OTHER, BAGS, and KEYS.

```

Choose > 2

+-----+
YOUR REPORTS
Use the full ID shown below to open a report.
+-----+
8 report(s)

LOST / OPEN   Blue steel bottle
ID: L001
OTHER | Central Library | 2026-09-01

FOUND / OPEN  Blue steel bottle
ID: F001
OTHER | Central Library | 2026-09-02

FOUND / OPEN  Blue bottle
ID: F002
OTHER | Cafeteria | 2026-09-03

LOST / OPEN   Black laptop sleeve
ID: L002
BAGS | Academic Block | 2026-09-02

FOUND / OPEN  Black laptop sleeve
ID: F003
BAGS | Academic Block | 2026-09-04
Enter = next page, q = stop listing >

LOST / RESOLVED Room keys
ID: L003
KEYS | Hostel Gate | 2026-09-01

FOUND / RESOLVED Room keys
ID: F004
KEYS | Hostel Gate | 2026-09-01

```

Figure 9. The report list (option 2), paginated five records at a time with type, status, title, ID, category, location, and date.

9.4 Opening a single report

Entering a report ID such as l003 (case-insensitive) at the manage prompt opens that report's full detail view, including its description and contact reference, with a prompt to reopen it for matching.

```

+-----+
HOME
Report. Search. Reconnect.
+-----+
1 Add a report
2 View & manage reports
3 Find potential matches
4 Search reports
5 Statistics
0 Exit
Choose > 3
Report ID (blank = back) > l003

+-----+
ROOM KEYS
LOST / RESOLVED
+-----+
ID: L003
Category: KEYS
Location: Hostel Gate
Incident date: 2026-09-01
Description: Two silver keys on green ring
Contact / reference: Demo desk F
Reopen this report to look for matches.

```

Figure 10. Report detail view for L003 ("Room keys"), reached from the home menu via option 2 then option 3.

9.5 Searching reports

Option 4 combines multiple filters — text, location, category, type, and status — applied together. In this example the filters (title contains "bottle", location contains "ab2", category OTHER, type LOST, status OPEN) are narrow enough that no existing report matches, and the application prompts to try another filter or add a report.

```

+-----+
1 Add a report
2 View & manage reports
3 Find potential matches
4 Search reports
5 Statistics
0 Exit
Choose > 4

+-----+
SEARCH
Leave a filter blank to include any value. Dates are inclusive.
+-----+
Title / description contains > bottle
Location contains > ab2
[ELECTRONICS, ID_CARD, KEYS, BOOKS, CLOTHING, BAGS, OTHER]
Category (blank = any) > other
[LOST, FOUND]
Type (blank = any) > lost
[OPEN, RESOLVED]
Status (blank = any) > open
From (YYYY-MM-DD, blank = any) >
To (YYYY-MM-DD, blank = any) >
No reports to show. Try another filter or add a report.

```

Figure 11. Combined search filters (option 4) returning no results for an overly narrow query.

9.6 Statistics

Option 5 prints the campus snapshot: total report count plus breakdowns by type, status, and category, reflecting the report added earlier in the session.

```
+-----+
HOME
Report. Search. Reconnect.
+-----+
1 Add a report
2 View & manage reports
3 Find potential matches
4 Search reports
5 Statistics
0 Exit
Choose > 5

+-----+
CAMPUS SNAPSHOT
Counts describe reports, not verified item returns.
+-----+
Total reports: 8
LOST: 3
FOUND: 5
OPEN: 6
RESOLVED: 2
ELECTRONICS: 0
ID_CARD: 0
KEYS: 2
BOOKS: 0
CLOTHING: 0
BAGS: 2
OTHER: 4
```

Figure 12. Statistics view (option 5) — 8 total reports: 3 LOST, 5 FOUND, 6 OPEN, 2 RESOLVED.

9.7 Exit

Choosing option 0 from the home menu closes the application cleanly, confirming that saved reports will be available on the next run.

```
+-----+
HOME
Report. Search. Reconnect.
+-----+
1 Add a report
2 View & manage reports
3 Find potential matches
4 Search reports
5 Statistics
0 Exit
Choose > 0
Your saved reports will be here next time. Goodbye.
```

Figure 13. Exiting the application (option 0); all changes made during the session are already persisted.

Together these captures demonstrate the full functional loop — create, list, inspect, search, and summarise — matching the workflow described in Section 6.2 and the functional requirements in Section 3.

10. Testing Approach

All automated checks are in test/ProjectTest.java. The test scripts compile the application and this file before running the checks. A failed assertion stops the run with a non-zero exit code. Tests create temporary datasets, including for subprocesses that exercise the real terminal entry point.

Test group	Checks	Coverage
Matching	19	Eligibility, scoring, boundaries, token normalization, empty sets, tie

Test group	Checks	Coverage
		ordering
Input validation	14	Blank / long / control input, leap days, future dates, enums, normalization
CSV storage	14	UTF-8 quoting, restart, malformed data, case-insensitive duplicate IDs, locks, rejected-save preservation
Report operations	16	CRUD, filters, dates, status, counts, immutable snapshots, failed-save memory rollback
CLI workflows	13	Create / edit / search / resolve / reopen / delete flows, cancellation, EOF, bad options, plain mode
Synthetic benchmark	1	Correct result count for 10,000 eligible candidates

The revised package was tested on 17 September 2026 using Microsoft OpenJDK 21.0.12 on Windows. All 77 checks passed. The 10,000-candidate matching run took 118.46 ms, well below the 2-second target. Application and test code also compiled with `--release 21 -Xlint:all` without warnings. Run `test.cmd` on Windows or `sh test.sh` on macOS/Linux to print the individual results.

The benchmark measures matching for one report; it excludes file loading and saving. A single timing sample cannot predict performance on every computer. The tests check specified behaviours, not the accuracy of real-world ownership suggestions. The macOS/Linux scripts are supplied but have not been run on those operating systems.

11. Challenges Faced

- CSV fields can contain commas and quotation marks. Splitting every row on commas would break those values, so FileManager reads quoted fields character by character and doubles internal quotes when saving. Storage tests include commas, quotes, and accented characters.
- A save can fail after a user has requested a change. ReportService keeps the old list until the replacement file is written successfully; one test deliberately places a non-empty directory at the CSV path to trigger a write failure and confirms the previous records are kept.
- Two reports may share a category and location while describing different items. The matcher displays the separate score components and shared keywords so the operator can still compare details and confirm ownership before returning an item.
- Terminal rendering varies between environments. ConsoleUI keeps colour and border formatting in one place, and `--plain` produces ASCII output without ANSI sequences; scripted input allows complete menu workflows to be tested automatically.

- ID comparison needed to be consistent throughout the application. An earlier review found that IDs such as L001 and l001 could be loaded together even though lookup treated them as the same ID. Loading, saving, and matching now follow the same case-insensitive rule, covered by three regression checks.

12. Learnings & Key Takeaways

- Separating responsibilities makes a small program easier to understand. Models describe valid data, the service manages changes, storage handles files, and ConsoleUI handles interaction — matching rules can be exercised without a terminal session or storage dependency.
- Sets fit the keyword-comparison problem because duplicates should not change the score. The intersection records shared words, while the union records all distinct words across both reports.
- Persistence requires checking failure cases as well as successful saves. Reloading records, testing quoted text, and forcing a write failure reveal problems that a normal demonstration may miss.
- A validation rule should mean the same thing at every layer — case-insensitive lookup must be paired with case-insensitive duplicate checks, with boundary tests making such rules explicit.
- A numerical score is more useful when its meaning is clear. The application shows the evidence behind a suggestion and leaves the ownership decision to a person; the local storage design also defines a practical limit on multi-computer collaboration.

13. Future Enhancements

- Campus location aliases mapping common abbreviations to a standard name, tested carefully for ambiguous aliases.
- Measuring matching quality using consented, anonymised report pairs with manually verified outcomes, comparing weights and thresholds by precision and recall.
- A verified-return operation linking a lost report to a found report after ownership is checked, updating both statuses together with a resolution reference.
- A shared service and database for access from several computers, which would also require authentication, permissions, privacy decisions, and safe handling of concurrent changes.
- Optional backups, an audit trail, student usability feedback, and testing on additional operating systems.

14. References

1. [VITYarthi — Build Your Own Project: General Project Instructions & Submission Guidelines](#)

2. VITyarthi course portal — project-submission instructions: repository visibility, repository URL format, terminal execution, report submission, originality, and evaluation policy.

3. [Oracle — Java Platform, Standard Edition 21: The javac Command](#)

4. [Oracle — Java SE 21 API Specification: LocalDate](#)

5. [Oracle — Java SE 21 API Specification: Files](#)

6. Project materials — GitHub repository: <https://github.com/YatharthSingh-VIT/campus-lost-found-matcher>

Application source under src/com/lostfound; automated checks in test/ProjectTest.java; fictional sample records in examples/items.csv; setup and testing instructions in README.md.